

Vendor management and HRMS systemFree plan · Upgrade

I have a requirement to make an HRMS and vendor management kind of system. The situation is like this: one PSU company hires vendor employees through NICSi, and they want a way to manage purchase orders, Work orders, Advance payments, vendor employee details, and attendance.
It happens like this: Suppose there are multiple projects in the company and there is a requirement for more manpower, so the project manager can raise a purchase order request through SAP, and the purchase department will handle the purchase order process; we only need to

Show more

Architected comprehensive system design encompassing data models and workflows

Manpower r
PO raised in SAP

↓ t
SU

Want to be notified when Claude responds?

Notify

Write a message...

+Sonnet 5 Medium

Claude is AI and can make mistakes. Please double-check responses.

Nicsi vendor hrms design · MDCopy

PSU-NICSi Vendor Management & HRMS — System Design

1. Purpose

Track the full lifecycle of vendor manpower sourced through NICSi; purchase orders, work orders, advance payments, TDS, vendor employee master data, project-wise assignment, attendance, and NICSi advance reconciliation — with two roles (Admin, Project Manager) and dashboards for advance and attendance monitoring.

2. Core entities and relationships

NICSi Subunit (1) —< Project (M)
User/PM (1) —< Project (M)
Project (1) —< Purchase Order (M)
Purchase Order (1) —< Work Order (1) [strict 1:1]
Work Order (1) —< Advance (1) [strict 1:1]
Advance (1) —< Invoice (M) [each invoice reduces balance]
Work Order (1) —< Assignment (M) —> Vendor Employee (M)
Project (1) —< Assignment (M)
Assignment (1) —< Attendance (M, daily)
Project (1) —< Attendance Certificate (M, monthly)

2.1 Entity details

NICSi_Subunit

- `nicsi_project_id` (PK, given by NICSi, one per PSU subunit)
- `subunit_name`
- Purpose: the aggregation point for "overall advance with NICSi across all projects."

Project (PSU-internal, multiple per subunit)

- `project_id` (PK, PSU-generated), `project_name`, `description`
- `nicsi_subunit_id` (FK)
- `project_manager_id` (FK → User)
- `status` (Active/Closed)

PurchaseOrder

- `po_no` (PK, from SAP), `project_id` (FK), `po_date`
- `manpower_count_requested`, `duration`, `remarks`
- `status` : Draft → Advance Pending → Advance Paid → WO Issued → Closed

WorkOrder (1:1 with PO)

- `wo_no` (PK, from NICSi), `po_no` (FK, unique), `project_id` (FK, denormalized for query speed)
- `start_date`, `end_date` (validity/expiry — drives renewal alerts)
- `total_value`, `status` : Active / Expiring / Expired / Closed

Advance (1:1 with WO)

- `advance_id` (PK), `wo_no` (FK, unique), `po_no` (FK), `project_id` (FK)
- `gross_amount` (e.g. 12,00,000), `tds_amount`, `net_paid`
- `payment_ref` (bank UTR or cheque number), `payment_date`
- `balance_amount` (decrements as invoices arrive)
- `status` : Active / Low-balance / Exhausted / Closed

Invoice (from NICSi, monthly, tied to attendance)

- `invoice_no` (PK), `advance_id` (FK), `wo_no` (FK)
- `month_year`, `invoice_amount`, `attendance_cert_id` (FK)
- `balance_after` (snapshot of Advance.balance_amount post-deduction)

AttendanceCertificate

- `cert_id` (PK), `project_id` (FK), `wo_no` (FK), `month_year`
- `total_present_days`, `generated_by`, `sent_to_nicsi_date`, `file_ref` (PDF)

Attendance (daily)

- `attendance_id` (PK), `assignment_id` (FK), `date`, `status` (Present/Absent/Leave/Half-day), `remarks`

VendorEmployee (master data)

- `emp_id` (PK), `name`, `dob`, `contact`, `cv_file`, `pf_no`, `esi_insurance_no`
- `agency_name` (the NICSi-empanelled agency), `bank_details`, `joining_date`, `status`

Assignment (junction — an employee's posting to a specific WO/project, historized)

- `assignment_id` (PK), `emp_id` (FK), `wo_no` (FK), `project_id` (FK)
- `designation`, `monthly_salary`, `start_date`, `end_date`, `status`

User

- `user_id`, `name`, `email`, `role` (Admin / Project Manager)

2.2 Why Assignment is separate from VendorEmployee

A vendor employee's CV/PF/insurance is filed once as master data. Their actual posting — which project, which WO, what salary, what dates — changes over the person's tenure (e.g. moved from an expired WO to a renewed one, or shifted between projects). Modeling this as its own table gives you full history instead of overwriting a single "current project" field, and lets attendance and invoice consumption trace back to the exact WO/advance that funded it.

3. Workflow (lifecycle)

1. **Manpower requirement** → Project Manager raises request in SAP → PO number generated by Purchase dept. PSU system records `po_no` + basic details only (status: Draft).
2. **CV sourcing & selection** → NICSI shares CVs → PSU interviews → candidates selected. (Optional: a lightweight `Candidate` table linked to the PO — Shortlisted/Selected/Rejected — if you want this stage tracked rather than just noted informally.)
3. **Pricing & advance** → NICSI quotes price → PSU computes advance (headcount × salary × duration + TDS) → Advance paid, recorded with bank ref/cheque no, TDS, and date. PO status → Advance Paid.
4. **Work order** → NICSI issues WO against the advance → recorded 1:1 against the PO, with resource details and validity dates. WO status → Active.
5. **Onboarding** → Vendor employees' master data entered (or reused if already in system) → `Assignment` record links them to this WO + Project.
6. **Attendance** → Daily attendance logged per assignment.
7. **Monthly certificate** → PSU issues one Attendance Certificate per project/WO to NICSI.
8. **Invoice & reconciliation** → NICSI (via the agency) processes salary, then invoices the PSU against the certificate → `Invoice` recorded → `Advance.balance_amount` reduced.
9. **Renewal** → When WO expires or balance nears zero while manpower is still needed → new PO → new WO → new Advance, repeating under the same `Project`. Old WO/advance closed, historical trail preserved.

3.1 Status state machines (suggested)

- PO: `Draft` → `Advance Paid` → `WO Issued` → `Closed`
- WO: `Active` → `Expiring` (e.g. 30 days out) → `Expired` → `Closed`
- Advance: `Active` → `Low-balance` (below a configurable threshold, e.g. 15%) → `Exhausted` → `Closed`

4. Roles & permissions

Action	Admin	Project Manager
Create/manage users	Yes	No
Create/edit any project	Yes	No (view/manage only own)
Create PO, WO, Advance entries	Yes	Yes (own projects)
Vendor employee master data	Yes	Yes (add/edit for own projects)
Mark attendance	Yes	Yes (own projects)
Generate attendance certificate	Yes	Yes (own projects)
View dashboards	All projects	Own projects only
NICSI subunit-level aggregate advance	Yes	No (project-level only)

A PM can be assigned multiple projects (one-to-many: Project → PM). Consider a lightweight **maker-checker** step for Advance entries specifically (PM enters, Admin confirms) — financial records like bank references and TDS are the most audit-sensitive part of this system, even though it's not explicitly required.

5. Dashboards

Advance dashboard

- Per-project: advance paid, consumed (via invoices), balance, % consumed
- Aggregate: total advance with NICSI across all projects (rolled up via NICSI Subunit)

- Alerts: WOs nearing expiry, advances below low-balance threshold
- Drill-down: PO → WO → Advance → Invoice trail

Attendance dashboard

- Monthly attendance % per project / per employee
- Present/Absent/Leave trends
- Pending attendance certificates (not yet sent to NICSI)

PO/WO tracking dashboard

- Status funnel: Draft → Advance Paid → WO Issued → Closed
- List of expired WOs with active manpower still required (renewal candidates)

Vendor employee dashboard

- Headcount per project / per agency
- Expiring PF/insurance/document flags

6. Architecture recommendation

Stack

- Frontend: React (role-based routing; two roles keeps this simple)
- Backend: REST API — Node.js/NestJS or Spring Boot (Spring Boot is often preferred for PSU/ government audit and empanelment requirements — worth checking your PSU's IT policy)
- Database: PostgreSQL — relational integrity matters here (enforce the PO-WO 1:1 and WO-Advance 1:1 with unique foreign keys, not just application logic)
- File storage: CVs, certificates, invoices — since this is a PSU, check data-residency rules; on-prem storage or an empanelled government cloud (e.g. MeghRaj/NIC) may be mandated over public cloud
- Auth: JWT + RBAC (only 2 roles, so this stays simple)
- Notifications: email/SMS for WO expiry and low-advance alerts
- Audit log: every change to PO/WO/Advance/Invoice records who changed what and when — this is the kind of data a CAG or internal audit will want to trace

Non-functional notes

- Since NICSI gives one project ID per subunit, store it once at the subunit level and reference it from every project — don't duplicate it per project.
- Enforce PO↔WO and WO↔Advance as database-level unique constraints, not just UI validation.
- Keep Assignment historized (start/end dates) rather than a single current-project pointer on VendorEmployee, so past postings remain queryable after renewals.

7. Open questions worth deciding before build

- Do you want the CV-shortlisting stage (NICSI shares CVs → interview → selection) tracked as a formal `Candidate` entity, or just as an informal step before PO/Advance entry?
- Should Advance entries require Admin approval after PM entry (maker-checker), given the financial sensitivity of bank references and TDS?
- Threshold for "low-balance" advance alerts — fixed percentage, fixed amount, or configurable per project?
- Data residency requirement — any mandate for on-prem/government cloud hosting given PSU status?